

FILE: app.py

```
0001: from __future__ import annotations
0002:
0003: from pathlib import Path
0004:
0005: import streamlit as st
0006: from PIL import Image
0007:
0008: from scripts.build_index import build_reference_index, collect_images
0009: from src.breed_recognition import BreedRecognizer
0010:
0011:
0012: LANGUAGE_OPTIONS = {
0013:     "English": "en",
0014:     "????????????????": "hi",
0015:     "????????????????": "mr",
0016: }
0017:
0018: TEXT = {
0019:     "en": {
0020:         "page_title": "PASHUPAHECHAN",
0021:         "lang": "Language",
0022:         "hero_kicker": "Indian Livestock Vision",
0023:         "hero_title": "????????????????????????????",
0024:         "hero_copy": "Breed recognition for Indian cattle and buffaloes with fast offline predi
ctions for field teams, students, and livestock extension workers.",
0025:         "indexed_breeds": "Indexed Breeds",
0026:         "indexed_breeds_sub": "Breed folders currently ready for matching.",
0027:         "recognition_mode": "Recognition Mode",
0028:         "recognition_mode_sub": "Runs offline and remains usable even with poor connectivity.",
0029:         "index_status": "Index Status",
0030:         "index_status_sub": "Reference images are checked and indexed automatically.",
0031:         "rebuilt": "Rebuilt at startup",
0032:         "ready": "Ready for prediction",
0033:         "waiting": "Waiting for dataset",
0034:         "supported": "Supported breeds",
0035:         "supported_copy": "Starter support includes major Indian cattle and buffalo breeds.",
0036:         "workflow_note": "Add more images in data/reference_images/<breed_name>/. The app rebui
lds its breed index automatically on startup.",
0037:         "upload_title": "Upload and analyze",
0038:         "upload_copy": "Use a clear side-view or near-profile animal image. Similar lighting an
d less background clutter improve matching.",
0039:         "upload_label": "Upload image",
0040:         "prediction_count": "Number of predictions",
0041:         "upload_caption": "Uploaded field image",
0042:         "predict": "Predict Breed",
0043:         "spinner": "Analyzing image and comparing with reference breeds...",
```

[illegible]

```
0076: "upload_title": "????????????????? ?????????????? ?????????????? ?????? ??????????????????",
    ???",
0077: "upload_copy": "????????????????? ?????? ?????????? ?????????????? ?????? ??????????????????????
    ?????? ?????????????? ?????????????? ?????????????? ?????????????????? ?????? ?????? ??????????????????????
    ?????????? ?????????? ?????? ?????????????????????? ?????????????????? ?????????????????? ??????????????",
0078: "upload_label": "????????????????????? ?????????????????? ??????????????",
0079: "prediction_count": "????????????????? ?????????????????????????????????????????????? ??????????
    ?????????? ??????????",
0080: "upload_caption": "????????????????? ?????? ?????? ??????????????????????",
0081: "predict": "????????????? ??????????????????????????????",
0082: "spinner": "????????????????????? ?????? ?????????????????????????????? ?????????????? ?????? ???
    ?????? ?????? ?????? ?????????????????????????????? ?????????????????????? ?????? ?????????????????????? ?????? ?????
    ?????? ??????... ",
0083: "no_index": "????????????????? ?????????????????? ?????????????????????????????? ??????????????????????
    ? ?????????????? ?????????????????? data/reference_images/<breed_name>/ ?????????? ?????????????? ?????
    ? ?????????????????????????????? ?????????????????????? ?????? ?????? ?????????? ?????? ?????????????? ?????
    ??????????",
0084: "most_likely": "????????????? ?????????????????????????????? ??????????????????",
0085: "confidence": "????????????????????????????? ??????????????",
0086: "match_confidence": "????????????????????? ?????????????????????????????? ??????????????",
0087: "insight_title": "????????????????? ?????????? ?????????????????? ?????? ??????????????????",
0088: "upload_prompt": "????????????????? ?????????????? ?????????????? ?????? ?????????? ??????????
    ?????? ?????????????? ?????? ?????????????????????? ?????????????????? ??????????????????",
0089: "field_guidance": "????????????????????? ??????????????????",
0090: "field_guidance_copy": "?????? ?????????????????????????????????????? ?????? ?????????????? ?????????
    ?????????????? ?????????????? ?????????????????????????????????????? ?????? ?????????????????? ?????????????? ???
    ?????? ?????????????????? ?????????????????????????? ?????? ?????????? ?????? ?????????????? ??????????????????
    ??? ?????????? ?????? ?????????????? ?????????????????? ?????? ?????? ?????????? ??????????????????????????
    ??? ?????????????????? ?????????????????? ?????????????????????????? ?????? ?????????? ?????????????? ?????????????? ??
    ?????????? ?????????????????????? ?????????????????????????? ?????? ?????????? ?????????????? ??????????????
    ?????????????????????? ?????????????????? ?????? ?????????????? ??????????",
0091: "offline_cnn": "????????????????????? ?????????? ?????????????????????? ??????: ?????????????? CN
    N ?????????????? ?????????????????????????????????????????????? ?????????? ?????? ?????????? ??????????",
0092: "offline_hand": "????????????????????? ?????????? ?????????????????????? ??????: ?????????????????
    ?-?????? ?????????????????????????????????????????????? ?????????????? ?????????????? ?????????????????? ???
    ?? ?????????? ?????????????? ?????????????????????????? ?????? ?????????????????? ?????????????? ??????????",
0093: "backend_cnn": "??????????????? CNN ??????????????????????????????????????????????",
0094: "backend_hand": "????????????????????????????????????????????? ?????????????????????? ?????????????
    ??????????????????????????????????????",
0095: "breed_fallback": "??????????????? ?????????? ?????????? ?????? ?????????????? ??????????????????
    ?????? ?????????????????? ?????? ?????????????? ??????????",
0096: },
0097: "mr": {
0098: "page_title": "?????????????????????????????????",
0099: "lang": "?????????????",
0100: "hero_kicker": "????????????????????? ?????????????????? ??????????????????????",
0101: "hero_title": "?????????????????????????????????",
0102: "hero_copy": "????????????????????????? ?????????????? ?????????? ??????????????????????????????????????????"
```

[illegible]

```
0130:         "offline_cnn": "???????????????????? ?????????? ??????????????????????: ?????????????? CNN ?????
????????? ?????????????????????????????????????????????????????????????? ?????????? ?????????????? ??????????.",
0131:         "offline_hand": "???????????????????? ?????????? ??????????????????????: ?????????????????????? ??
????????????????????????????????????????????????????????????? ?????????????? ?????????????????????? ??????????
????????????????? ?????????????????????????????????????????? ?????????????? ??????????????????.",
0132:         "backend_cnn": "??????????????? CNN ??????????????????????????????????????????????????????????",
0133:         "backend_hand": "????????????????????????????????????????????????????????????? ?????????????????????? ??????????????????
?????????????????????????????????????????",
0134:         "breed_fallback": "??????????????? ???????????????? ???????????????? ?????????????????????? ?????????????
????????? ?????????????????????? ??????????????????.",
0135:     },
0136: }
0137:
0138: INDIAN_CATTLE_BREEDS = [
0139:     "Gir",
0140:     "Sahiwal",
0141:     "Rathi",
0142:     "Tharparkar",
0143:     "Red Sindhi",
0144:     "Kankrej",
0145:     "Ongole",
0146:     "Haryana",
0147:     "Hallikar",
0148:     "Deoni",
0149: ]
0150:
0151: INDIAN_BUFFALO_BREEDS = [
0152:     "Murrah",
0153:     "Jaffarabadi",
0154:     "Mehsana",
0155:     "Bhadawari",
0156:     "Surti",
0157:     "Nili-Ravi",
0158:     "Pandharpuri",
0159:     "Nagpuri",
0160:     "Toda",
0161: ]
0162:
0163: BREED_DETAILS = {
0164:     "Gir": "Heat-tolerant dairy breed from Gujarat, known for a domed forehead and long ears.",
0165:     "Sahiwal": "Strong milch breed with a reddish-brown coat, common in North India and Punjab
regions.",
0166:     "Rathi": "Dual-purpose cattle from Rajasthan, valued for both milk yield and hardiness.",
0167:     "Tharparkar": "Desert-adapted breed recognized for endurance and pale grey-white coloring."
```

```

0168:     "Red Sindhi": "Deep reddish dairy breed suited to hot climates and low-input systems.",
0169:     "Kankrej": "Large and powerful breed from western India, often used for both milk and draught work.",
0170:     "Ongole": "Tall, muscular cattle breed from Andhra Pradesh with prominent hump structure.",
0171:     "Hariana": "North Indian breed appreciated for draught strength and moderate milk production.",
0172:     "Hallikar": "Classic southern draught breed, usually lean, agile, and energetic.",
0173:     "Deoni": "Spotted dual-purpose cattle from the Deccan region, productive under village conditions.",
0174:     "Murrah": "Premier Indian buffalo breed, jet black, tightly curled horns, and high milk potential.",
0175:     "Jaffarabadi": "Heavy buffalo breed with massive body frame and drooping horns.",
0176:     "Mehsana": "Buffalo breed from Gujarat, often selected for stable milk yield.",
0177:     "Bhadawari": "Copper-toned buffalo known for rich-fat milk and adaptation to hot plains.",
0178:     "Surti": "Medium-sized buffalo with sickle-shaped horns and a calm dairy temperament.",
0179:     "Nili-Ravi": "Large river buffalo breed with a strong dairy profile and characteristic white markings.",
0180:     "Pandharpuri": "Maharashtra buffalo breed with long sword-like horns and elongated face.",
0181:     "Nagpuri": "Hardy buffalo breed used in dry regions, recognized for long flat horns.",
0182:     "Toda": "Nilgiri hill buffalo associated with the Toda community and adapted to upland conditions.",
0183: }
0184:
0185: BREED_INSIGHTS = {
0186:     "Gir": {
0187:         "trait": "Long pendulous ears with a prominent domed forehead are strong visual markers.",
0188:         "value": "Well known for dairy strength and heat tolerance in hot regions.",
0189:     },
0190:     "Sahiwal": {
0191:         "trait": "Reddish coat tone with a deep body and calm dairy-type build.",
0192:         "value": "One of the most respected indigenous dairy breeds in the subcontinent.",
0193:     },
0194:     "Murrah": {
0195:         "trait": "Jet-black body with tightly curled horns is the classic Murrah marker.",
0196:         "value": "Top Indian buffalo breed for high milk production potential.",
0197:     },
0198:     "Kankrej": {
0199:         "trait": "Large, powerful frame with a characteristic lyre-shaped horn profile.",
0200:         "value": "Strong dual-purpose breed with both milk and draught importance.",
0201:     },
0202:     "Pandharpuri": {
0203:         "trait": "Very long sword-like horns make this buffalo visually striking.",
0204:         "value": "Adapted to regional conditions in Maharashtra.",
0205:     },
0206: }
0207:
0208: REFERENCE_ROOT = Path("data/reference_images")

```

```

0209: INDEX_PATH = Path("models/reference_index.json")
0210:
0211:
0212: def needs_index_rebuild(reference_root: Path, index_path: Path) -> bool:
0213:     if not index_path.exists():
0214:         return True
0215:     image_files = collect_images(reference_root) if reference_root.exists() else []
0216:     if not image_files:
0217:         return False
0218:     latest_image_time = max(path.stat().st_mtime for path in image_files)
0219:     return index_path.stat().st_mtime < latest_image_time
0220:
0221:
0222: def ensure_reference_index() -> tuple[bool, int]:
0223:     if not REFERENCE_ROOT.exists():
0224:         return False, 0
0225:     image_files = collect_images(REFERENCE_ROOT)
0226:     if not image_files:
0227:         return False, 0
0228:     if needs_index_rebuild(REFERENCE_ROOT, INDEX_PATH):
0229:         index = build_reference_index(REFERENCE_ROOT, INDEX_PATH)
0230:         return True, len(index)
0231:     recognizer = BreedRecognizer(index_path=str(INDEX_PATH))
0232:     return False, len(recognizer.reference_vectors)
0233:
0234:
0235: def render_hero_art() -> str:
0236:     return ""
0237:     <svg viewBox="0 0 640 420" xmlns="http://www.w3.org/2000/svg" role="img" aria-label="Cow standing in an Indian farm field">
0238:         <defs>
0239:             <linearGradient id="sky" x1="0" x2="0" y1="0" y2="1">
0240:                 <stop offset="0%" stop-color="#82cfff"/>
0241:                 <stop offset="100%" stop-color="#fff6dd"/>
0242:             </linearGradient>
0243:             <linearGradient id="hill" x1="0" x2="1" y1="0" y2="1">
0244:                 <stop offset="0%" stop-color="#90c56d"/>
0245:                 <stop offset="100%" stop-color="#4f9650"/>
0246:             </linearGradient>
0247:             <linearGradient id="field" x1="0" x2="1" y1="0" y2="1">
0248:                 <stop offset="0%" stop-color="#d2b25f"/>
0249:                 <stop offset="100%" stop-color="#7ca348"/>
0250:             </linearGradient>
0251:             <linearGradient id="cowBody" x1="0" x2="1" y1="0" y2="1">
0252:                 <stop offset="0%" stop-color="#fff7ef"/>
0253:                 <stop offset="100%" stop-color="#f1ddc8"/>
0254:             </linearGradient>
0255:         </defs>

```

```

0256:      <rect width="640" height="420" rx="34" fill="url(#sky)"/>
0257:      <circle cx="528" cy="84" r="42" fill="#ffd25e"/>
0258:      <path d="M0 236C104 194 185 220 254 206C340 189 401 145 484 156C552 166 600 206 640 194V4
20H0Z" fill="url(#hill)"/>
0259:      <path d="M0 295C88 279 144 306 230 295C305 286 360 250 428 248C511 246 568 284 640 270V42
0H0Z" fill="url(#field)"/>
0260:      <path d="M0 330C81 311 145 340 210 322C278 303 334 280 420 293C508 307 566 344 640 325V42
0H0Z" fill="#6d9d43" opacity="0.75"/>
0261:      <g transform="translate(150 165)">
0262:          <ellipse cx="158" cy="120" rx="132" ry="78" fill="url(#cowBody)"/>
0263:          <ellipse cx="259" cy="86" rx="56" ry="49" fill="url(#cowBody)"/>
0264:          <path d="M248 51L276 28L271 62Z" fill="#be8750"/>
0265:          <path d="M224 54L193 30L207 68Z" fill="#be8750"/>
0266:          <circle cx="236" cy="82" r="5.5" fill="#182447"/>
0267:          <path d="M116 82C130 59 171 59 184 83C165 99 136 104 116 82Z" fill="#c98546"/>
0268:          <path d="M172 128C187 111 223 112 233 142C212 157 184 155 172 128Z" fill="#c98546"/>
0269:          <path d="M87 137C97 118 125 116 140 135C123 149 100 151 87 137Z" fill="#c98546"/>
0270:          <rect x="96" y="191" width="18" height="108" rx="9" fill="#7b5b44"/>
0271:          <rect x="158" y="193" width="18" height="106" rx="9" fill="#7b5b44"/>
0272:          <rect x="236" y="186" width="18" height="113" rx="9" fill="#7b5b44"/>
0273:          <rect x="286" y="182" width="18" height="117" rx="9" fill="#7b5b44"/>
0274:      </g>
0275:  </svg>
0276:  ""
0277:
0278:
0279: def inject_styles() -> None:
0280:     st.markdown(
0281:         ""
0282:         <style>
0283:             .stApp {
0284:                 background:
0285:                     radial-gradient(circle at top right, rgba(244, 185, 66, 0.30), transparent 26%)
0286:                     ,
0287:                     radial-gradient(circle at left 18%, rgba(75, 163, 242, 0.18), transparent 24%),
0288:                     radial-gradient(circle at bottom right, rgba(217, 79, 112, 0.16), transparent 2
0289:                     2%),
0290:                     linear-gradient(180deg, #fff7e7 0%, #fffd7 46%, #f8f0ff 100%);
0291:             }
0292:             .block-container {
0293:                 padding-top: 1.5rem;
0294:                 padding-bottom: 3rem;
0295:             }
0296:             .hero-shell, .panel-shell, .metric-card, .hero-art-shell, .result-hero, .insight-card {
0297:                 border-radius: 24px;
0298:                 border: 1px solid rgba(27, 36, 64, 0.09);
0299:                 box-shadow: 0 12px 32px rgba(29, 41, 64, 0.08);
0300:             }

```



```
0299:         .hero-shell, .panel-shell, .metric-card {
0300:             background: linear-gradient(180deg, rgba(255,255,255,0.92), rgba(255,248,236,0.86))
;
0301:             padding: 1.3rem;
0302:         }
0303:         .hero-art-shell {
0304:             background: linear-gradient(180deg, rgba(255,255,255,0.96), rgba(255,248,236,0.90))
;
0305:             padding: 0.7rem;
0306:         }
0307:         .hero-kicker {
0308:             display: inline-block;
0309:             padding: 0.4rem 0.8rem;
0310:             border-radius: 999px;
0311:             background: linear-gradient(135deg, rgba(47,125,79,0.14), rgba(75,163,242,0.12));
0312:             font-size: 0.8rem;
0313:             font-weight: 800;
0314:             letter-spacing: 0.14em;
0315:             text-transform: uppercase;
0316:             color: #2f7d4f;
0317:         }
0318:         .hero-title {
0319:             font-size: clamp(2.2rem, 4vw, 4rem);
0320:             font-weight: 800;
0321:             color: #17213d;
0322:             margin: 0.8rem 0;
0323:         }
0324:         .hero-copy, .section-copy, .metric-sub, .result-copy, .insight-value {
0325:             color: #5d6486;
0326:             line-height: 1.6;
0327:         }
0328:         .metric-label, .result-label, .insight-title {
0329:             color: #5d6486;
0330:             font-size: 0.82rem;
0331:             text-transform: uppercase;
0332:             letter-spacing: 0.12em;
0333:             font-weight: 700;
0334:         }
0335:         .metric-value, .result-breed {
0336:             color: #182447;
0337:             font-weight: 800;
0338:         }
0339:         .metric-value {
0340:             font-size: 1.7rem;
0341:             margin-top: 0.35rem;
0342:         }
0343:         .section-title {
0344:             font-size: 1.1rem;
```

```

0345:         font-weight: 800;
0346:         color: #1b2440;
0347:     }
0348:     .breed-chip {
0349:         display: inline-block;
0350:         margin: 0.25rem 0.35rem 0 0;
0351:         padding: 0.45rem 0.7rem;
0352:         border-radius: 999px;
0353:         background: linear-gradient(135deg, rgba(75,163,242,0.14), rgba(217,79,112,0.14));
0354:         color: #1b2440;
0355:         font-size: 0.88rem;
0356:         font-weight: 600;
0357:     }
0358:     .workflow-note {
0359:         margin-top: 0.8rem;
0360:         padding: 0.85rem 1rem;
0361:         border-left: 4px solid #ef6a3c;
0362:         background: linear-gradient(135deg, rgba(239,106,60,0.10), rgba(244,185,66,0.12));
0363:         border-radius: 12px;
0364:     }
0365:     .result-hero {
0366:         background: linear-gradient(135deg, rgba(47,125,79,0.15), rgba(244,185,66,0.24), rgba(75,163,242,0.14));
0367:         padding: 1.2rem 1.25rem;
0368:         margin-bottom: 1rem;
0369:     }
0370:     .insight-card {
0371:         background: linear-gradient(135deg, rgba(75,163,242,0.14), rgba(244,185,66,0.22), rgba(47,125,79,0.14));
0372:         padding: 1rem 1.1rem;
0373:         margin: 0.85rem 0 1rem 0;
0374:     }
0375:     .stButton > button {
0376:         background: linear-gradient(135deg, #ef6a3c, #d94f70, #f4b942);
0377:         color: white;
0378:         border: none;
0379:         border-radius: 14px;
0380:         font-weight: 800;
0381:     }
0382:     div[data-testid="stProgressBar"] > div > div > div {
0383:         background: linear-gradient(90deg, #2f7d4f, #4ba3f2, #f4b942);
0384:     }
0385: </style>
0386: "",
0387: unsafe_allow_html=True,
0388: )
0389:
0390:

```

```

0391: st.set_page_config(page_title="????????????????????", layout="wide")
0392: inject_styles()
0393:
0394: language_name = st.selectbox("Language", options=list(LANGUAGE_OPTIONS.keys()), index=0)
0395: t = TEXT[LANGUAGE_OPTIONS[language_name]]
0396:
0397: rebuilt_index, indexed_breeds = ensure_reference_index()
0398:
0399: if "recognizer" not in st.session_state:
0400:     st.session_state.recognizer = BreedRecognizer(index_path=str(INDEX_PATH))
0401: elif rebuilt_index:
0402:     st.session_state.recognizer = BreedRecognizer(index_path=str(INDEX_PATH))
0403:
0404: recognizer: BreedRecognizer = st.session_state.recognizer
0405: backend_label = t["backend_cnn"] if recognizer.backend == "offline-cnn" else t["backend_hand"]
0406: index_status = t["rebuilt"] if rebuilt_index else t["ready"] if indexed_breeds > 0 else t["waiting"]
0407:
0408: hero_col, art_col = st.columns([1.2, 0.95], gap="large")
0409: with hero_col:
0410:     st.markdown(
0411:         f"""
0412:         <div class="hero-shell">
0413:             <div class="hero-kicker">{t["hero_kicker"]}</div>
0414:             <div class="hero-title">{t["hero_title"]}</div>
0415:             <div class="hero-copy">{t["hero_copy"]}</div>
0416:         </div>
0417:         """,
0418:         unsafe_allow_html=True,
0419:     )
0420: with art_col:
0421:     st.markdown(f'<div class="hero-art-shell">{render_hero_art()}</div>', unsafe_allow_html=True)
0422:
0423: metric_col1, metric_col2, metric_col3 = st.columns(3)
0424: with metric_col1:
0425:     st.markdown(
0426:         f"""
0427:         <div class="metric-card">
0428:             <div class="metric-label">{t["indexed_breeds"]}</div>
0429:             <div class="metric-value">{indexed_breeds}</div>
0430:             <div class="metric-sub">{t["indexed_breeds_sub"]}</div>
0431:         </div>
0432:         """,
0433:         unsafe_allow_html=True,
0434:     )
0435: with metric_col2:
0436:     st.markdown(

```

```

0437:         f"""
0438:         <div class="metric-card">
0439:             <div class="metric-label">{t["recognition_mode"]}</div>
0440:             <div class="metric-value" style="font-size:1.25rem;">{backend_label}</div>
0441:             <div class="metric-sub">{t["recognition_mode_sub"]}</div>
0442:         </div>
0443:         """,
0444:         unsafe_allow_html=True,
0445:     )
0446: with metric_col3:
0447:     st.markdown(
0448:         f"""
0449:         <div class="metric-card">
0450:             <div class="metric-label">{t["index_status"]}</div>
0451:             <div class="metric-value" style="font-size:1.25rem;">{index_status}</div>
0452:             <div class="metric-sub">{t["index_status_sub"]}</div>
0453:         </div>
0454:         """,
0455:         unsafe_allow_html=True,
0456:     )
0457:
0458: if recognizer.backend == "offline-cnn":
0459:     st.success(t["offline_cnn"])
0460: else:
0461:     st.warning(t["offline_hand"])
0462:
0463: main_col, side_col = st.columns([1.45, 0.9], gap="large")
0464:
0465: with side_col:
0466:     st.markdown(f'<div class="panel-shell">', unsafe_allow_html=True)
0467:     st.markdown(f'<div class="section-title">{t["supported"]}</div>', unsafe_allow_html=True)
0468:     st.markdown(f'<div class="section-copy">{t["supported_copy"]}</div>', unsafe_allow_html=True)
0469:     st.markdown('".join(f'<span class="breed-chip">{breed}</span>' for breed in INDIAN_CATTLE_BREEDS), unsafe_allow_html=True)
0470:     st.markdown('".join(f'<span class="breed-chip">{breed}</span>' for breed in INDIAN_BUFFALO_BREEDS), unsafe_allow_html=True)
0471:     st.markdown(f'<div class="workflow-note">{t["workflow_note"]}</div>', unsafe_allow_html=True)
0472:     st.markdown("</div>", unsafe_allow_html=True)
0473:
0474: with main_col:
0475:     st.markdown(f'<div class="panel-shell">', unsafe_allow_html=True)
0476:     st.markdown(f'<div class="section-title">{t["upload_title"]}</div>', unsafe_allow_html=True)
0477:     st.markdown(f'<div class="section-copy">{t["upload_copy"]}</div>', unsafe_allow_html=True)
0478:
0479:     uploaded = st.file_uploader(t["upload_label"], type=["jpg", "jpeg", "png", "webp"])

```

```

0480:     top_k = st.slider(t["prediction_count"], min_value=1, max_value=5, value=3)
0481:
0482:     if uploaded is not None:
0483:         image = Image.open(uploaded)
0484:         st.image(image, caption=t["upload_caption"], use_container_width=True)
0485:
0486:         if st.button(t["predict"], type="primary"):
0487:             with st.spinner(t["spinner"]):
0488:                 predictions = recognizer.predict(image, top_k=top_k)
0489:
0490:                 if not predictions:
0491:                     st.error(t["no_index"])
0492:                 else:
0493:                     best = predictions[0]
0494:                     insight = BREED_INSIGHTS.get(best.breed)
0495:                     st.markdown(
0496:                         f"""
0497:                         <div class="result-hero">
0498:                             <div class="result-label">{t["most_likely"]}</div>
0499:                             <div class="result-breed">{best.breed}</div>
0500:                             <div class="result-copy">
0501:                                 {t["confidence"]}: {best.confidence * 100:.1f}%<br>
0502:                                 {BREED_DETAILS.get(best.breed, t["breed_fallback"])}
0503:                             </div>
0504:                         </div>
0505:                         """,
0506:                         unsafe_allow_html=True,
0507:                     )
0508:
0509:                     if insight:
0510:                         st.markdown(
0511:                             f"""
0512:                             <div class="insight-card">
0513:                                 <div class="insight-title">{t["insight_title"]}</div>
0514:                                 <div class="insight-trait">{insight["trait"]}</div>
0515:                                 <div class="insight-value">{insight["value"]}</div>
0516:                             </div>
0517:                             """,
0518:                             unsafe_allow_html=True,
0519:                         )
0520:
0521:                     for idx, pred in enumerate(predictions, start=1):
0522:                         st.markdown(f"***{idx}. {pred.breed}***")
0523:                         st.progress(float(pred.confidence), text=f"{pred.confidence * 100:.1f}% {t[
'match_confidence']}]")
0524:                         detail = BREED_DETAILS.get(pred.breed)
0525:                         if detail:
0526:                             st.caption(detail)

```

```

0527:     else:
0528:         st.info(t["upload_prompt"])
0529:
0530:     st.markdown("</div>", unsafe_allow_html=True)
0531:
0532: st.markdown(
0533:     f"""
0534:     <div class="panel-shell" style="margin-top: 1.3rem;">
0535:         <div class="section-title">{t["field_guidance"]}</div>
0536:         <div class="section-copy">{t["field_guidance_copy"]}</div>
0537:     </div>
0538:     """,
0539:     unsafe_allow_html=True,
0540: )

```

FILE: src\breed_recognition.py

```

0001: from __future__ import annotations
0002:
0003: import json
0004: from dataclasses import dataclass
0005: from pathlib import Path
0006: from typing import Dict, List
0007:
0008: import numpy as np
0009: from PIL import Image, ImageFilter, ImageOps
0010:
0011: try:
0012:     import torch
0013:     import torchvision.models as models
0014:     import torchvision.transforms as transforms
0015: except Exception:
0016:     torch = None
0017:     models = None
0018:     transforms = None
0019:
0020:
0021: @dataclass
0022: class Prediction:
0023:     breed: str
0024:     confidence: float
0025:
0026:
0027: class BreedRecognizer:
0028:     def __init__(self, index_path: str = "models/reference_index.json") -> None:
0029:         self.index_path = Path(index_path)
0030:         self.reference_vectors: Dict[str, np.ndarray] = {}

```

```

0031:         self.backend = "offline-handcrafted"
0032:         self.device = "cpu"
0033:         self.model = None
0034:         self.transform = None
0035:         self._initialize_backend()
0036:         self._load_index()
0037:
0038:     def _initialize_backend(self) -> None:
0039:         if torch is None or models is None or transforms is None:
0040:             return
0041:
0042:         try:
0043:             self.device = "cuda" if torch.cuda.is_available() else "cpu"
0044:             model = models.efficientnet_b0(weights=None)
0045:             model.classifier = torch.nn.Identity()
0046:             model.eval()
0047:             model.to(self.device)
0048:             self.model = model
0049:             self.transform = transforms.Compose(
0050:                 [
0051:                     transforms.Resize((224, 224)),
0052:                     transforms.ToTensor(),
0053:                     transforms.Normalize(
0054:                         mean=[0.485, 0.456, 0.406],
0055:                         std=[0.229, 0.224, 0.225],
0056:                     ),
0057:                 ]
0058:             )
0059:             self.backend = "offline-cnn"
0060:         except Exception:
0061:             self.backend = "offline-handcrafted"
0062:             self.device = "cpu"
0063:             self.model = None
0064:             self.transform = None
0065:
0066:     def _load_index(self) -> None:
0067:         if not self.index_path.exists():
0068:             self.reference_vectors = {}
0069:             return
0070:
0071:         with self.index_path.open("r", encoding="utf-8") as f:
0072:             raw = json.load(f)
0073:             self.reference_vectors = {
0074:                 breed: np.array(vector, dtype=np.float32) for breed, vector in raw.items()
0075:             }
0076:
0077:     def is_ready(self) -> bool:
0078:         return len(self.reference_vectors) > 0

```

```

0079:
0080:     def extract_embedding(self, image: Image.Image) -> np.ndarray:
0081:         image = image.convert("RGB")
0082:         if self.backend == "offline-cnn" and self.model is not None and self.transform is not None:
0083:             tensor = self.transform(image).unsqueeze(0).to(self.device)
0084:             with torch.no_grad():
0085:                 features = self.model(tensor).squeeze(0).detach().cpu().numpy()
0086:                 return self._normalize(features)
0087:
0088:         return self._extract_handcrafted_features(image)
0089:
0090:     def _extract_handcrafted_features(self, image: Image.Image) -> np.ndarray:
0091:         resized = image.resize((192, 192))
0092:         rgb = np.asarray(resized, dtype=np.float32) / 255.0
0093:
0094:         hist_parts: List[np.ndarray] = []
0095:         for channel in range(3):
0096:             hist, _ = np.histogram(rgb[:, :, channel], bins=16, range=(0.0, 1.0))
0097:             hist_parts.append(hist.astype(np.float32))
0098:
0099:         gray = ImageOps.grayscale(resized)
0100:         edges = gray.filter(ImageFilter.FIND_EDGES)
0101:         edge_array = np.asarray(edges, dtype=np.float32) / 255.0
0102:         edge_hist, _ = np.histogram(edge_array, bins=16, range=(0.0, 1.0))
0103:
0104:         brightness_mean = np.array([edge_array.mean()], dtype=np.float32)
0105:         brightness_std = np.array([edge_array.std()], dtype=np.float32)
0106:
0107:         downsample = gray.resize((16, 16))
0108:         texture = np.asarray(downsample, dtype=np.float32).flatten() / 255.0
0109:
0110:         features = np.concatenate(
0111:             hist_parts
0112:             + [
0113:                 edge_hist.astype(np.float32),
0114:                 brightness_mean,
0115:                 brightness_std,
0116:                 texture.astype(np.float32),
0117:             ]
0118:         )
0119:         return self._normalize(features)
0120:
0121:     @staticmethod
0122:     def _normalize(vector: np.ndarray) -> np.ndarray:
0123:         norm = np.linalg.norm(vector) + 1e-10
0124:         return vector / norm
0125:

```



```

0126:     @staticmethod
0127:     def _cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
0128:         return float(np.dot(a, b) / ((np.linalg.norm(a) + 1e-10) * (np.linalg.norm(b) + 1e-10))
0129:     )
0130:     def predict(self, image: Image.Image, top_k: int = 3) -> List[Prediction]:
0131:         if not self.is_ready():
0132:             return []
0133:
0134:         query_embedding = self.extract_embedding(image)
0135:         scores: List[Prediction] = []
0136:         for breed, ref_vector in self.reference_vectors.items():
0137:             if ref_vector.shape != query_embedding.shape:
0138:                 continue
0139:             sim = self._cosine_similarity(query_embedding, ref_vector)
0140:             conf = max(0.0, min(1.0, (sim + 1.0) / 2.0))
0141:             scores.append(Prediction(breed=breed, confidence=conf))
0142:
0143:         scores.sort(key=lambda x: x.confidence, reverse=True)
0144:         return scores[:top_k]

```

FILE: scripts\build_index.py

```

0001: from __future__ import annotations
0002:
0003: import argparse
0004: import json
0005: from pathlib import Path
0006: from typing import Dict, List
0007:
0008: import numpy as np
0009: from PIL import Image
0010:
0011: from src.breed_recognition import BreedRecognizer
0012:
0013:
0014: def collect_images(folder: Path) -> List[Path]:
0015:     exts = {".jpg", ".jpeg", ".png", ".webp"}
0016:     return [p for p in folder.rglob("*") if p.suffix.lower() in exts]
0017:
0018:
0019: def build_reference_index(reference_root: Path, output_file: Path) -> Dict[str, List[float]]:
0020:     recognizer = BreedRecognizer(index_path="models/does_not_need_to_exist.json")
0021:
0022:     breed_vectors: Dict[str, np.ndarray] = {}
0023:     for breed_dir in sorted([p for p in reference_root.iterdir() if p.is_dir()]):
0024:         images = collect_images(breed_dir)

```

```

0025:         if not images:
0026:             continue
0027:
0028:         vectors: List[np.ndarray] = []
0029:         for image_path in images:
0030:             try:
0031:                 with Image.open(image_path) as img:
0032:                     vectors.append(recognizer.extract_embedding(img))
0033:             except Exception:
0034:                 continue
0035:
0036:         if not vectors:
0037:             continue
0038:
0039:         centroid = np.mean(np.stack(vectors, axis=0), axis=0)
0040:         centroid = centroid / (np.linalg.norm(centroid) + 1e-10)
0041:         breed_vectors[breed_dir.name] = centroid
0042:
0043:     serializable = {
0044:         breed: vector.astype(np.float32).tolist() for breed, vector in breed_vectors.items()
0045:     }
0046:     output_file.parent.mkdir(parents=True, exist_ok=True)
0047:     with output_file.open("w", encoding="utf-8") as f:
0048:         json.dump(serializable, f, indent=2)
0049:     return serializable
0050:
0051:
0052: def main() -> None:
0053:     parser = argparse.ArgumentParser(description="Build reference index for breed recognition."
0054: )
0055:     parser.add_argument(
0056:         "--reference-root",
0057:         type=Path,
0058:         default=Path("data/reference_images"),
0059:         help="Folder containing one subfolder per breed.",
0060:     )
0061:     parser.add_argument(
0062:         "--output",
0063:         type=Path,
0064:         default=Path("models/reference_index.json"),
0065:         help="Where to save the generated reference index.",
0066:     )
0067:     args = parser.parse_args()
0068:
0069:     if not args.reference_root.exists():
0070:         raise FileNotFoundError(f"Reference folder not found: {args.reference_root}")
0071:
0072:     index = build_reference_index(args.reference_root, args.output)

```

```
0072:     print(f"Saved {len(index)} breed prototypes to {args.output}")
0073:
0074:
0075: if __name__ == "__main__":
0076:     main()
```

FILE: scripts__init__.py

0001:

FILE: src__init__.py

0001:

FILE: requirements.txt

```
0001: streamlit==1.41.1
0002: torch>=2.2.0
0003: torchvision>=0.17.0
0004: Pillow>=10.3.0
0005: numpy>=1.26.0
```